

2025-26

Chapitre 5 : Les Listes

Programmation

UE-4 : Informatique

Licences

Mentions

Licence Sciences pour l'ingénieur 2^{ème}
année

Double Licence Sciences de la terre –
Physique 2^{ème} année

Licence Sciences pour la santé 3^{ème}
année

CHAPUIS Yves André
UNIVERSITE DE STRASBOURG

Cours librement inspiré du « Cours de Python » de Patrick FUCHS et
Pierre POULAIN, Université de Paris, 2021

1. Introduction

1.1. C'est quoi une liste ?

Une **liste** est une structure de données qui contient une série de valeurs.

Python autorise la construction de *liste* contenant des valeurs de types différents (par exemple entier et chaîne de caractères), ce qui leur confère une grande flexibilité.

Une *liste* est déclarée par une série de valeurs (n'oubliez pas les guillemets, simples ou doubles, s'il s'agit de chaînes de caractères) séparées par des virgules, et le tout encadré par des crochets [].

1.2. Exemple de liste

En voici quelques exemples de listes :

```
1 animaux = ['girafe ', 'tigre ', 'singé ', 'souris ']
2 tailles = [5, 2.5, 1.75, 0.15]
3 mixte = ['girafe ', 5, 'souris ', 0.15]
```

```
1 animaux
['girafe ', 'tigre ', 'singé ', 'souris ']
```

```
1 tailles
[5, 2.5, 1.75, 0.15]
```

```
1 mixte
['girafe ', 5, 'souris ', 0.15]
```

Lorsque l'on affiche une *liste*, Python la restitue telle qu'elle a été saisie.

2. Indice de liste

Un des gros avantages d'une *liste* est que vous pouvez appeler ses éléments par leur position. Ce numéro est appelé **indice** (ou *index*) de la *liste*.

Liste: ['girafe', 'tigre', 'singé', 'souris']

Indice: 0 1 2 3

Soyez très attentifs au fait que les **indices** d'une *liste* de *n* éléments commence à **0** et se termine à **n - 1**. Voyez l'exemple suivant :

```
1 animaux = ['girafe', 'tigre', 'singé', 'souris']
1 animaux[0]
'girafe'
```

```
1 animaux[1]
```

```
'tigre'
```

```
1 animaux[2]
```

```
'singe'
```

```
1 animaux[3]
```

```
'souris'
```

Par conséquent, si on appelle l'élément d'*indice* 4 de notre liste, Python renverra un message d'erreur :

```
1 animaux[4]
```

```
-----
IndexError                                Traceback (most recent call last)
<ipython-input-17-3b1dc0f433f7> in <module>
----> 1 animaux[4]

IndexError: list index out of range
```

N'oubliez pas ceci ou vous risquez d'obtenir des bugs inattendus !

3. Opérations sur les listes

3.1. Opérateur + de concaténation

Tout comme les *chaînes de caractères*, les *listes* supportent l'**opérateur + de concaténation**, ainsi que l'opérateur pour la duplication :

```
1 ani1 = ['girafe', 'tigre']
2 ani2 = ['singe', 'souris']
```

```
1 ani1 + ani2
```

```
['girafe', 'tigre', 'singe', 'souris']
```

```
1 ani1 * 3
```

```
['girafe', 'tigre', 'girafe', 'tigre', 'girafe', 'tigre']
```

L'**opérateur +** est très pratique pour **concaténer** deux *listes*.

Dans l'exemple ci-dessous, nous allons ajouter des éléments à une liste en utilisant l'**opérateur de concaténation +** :

- On crée tout d'abord une liste vide :

```
1 a = []
2
3 a
```

[]

- Puis, on ajoute deux éléments à cette liste créée, l'un après l'autre :

```
1 a = a + [15]
2
3 a
```

[15]

```
1 a = a + [-5]
2
3 a
```

[15, -5]

Grâce à l'**opérateur + de concaténation**, on peut donc ajouter des éléments à la *liste*, comme dans le cas de la *liste* $a = [15, -5]$.

3.2. Méthode `.append()`

Il existe une autre approche pour ajouter un élément dans une *liste*. Il s'agit de l'utilisation des **méthodes**, que nous apporterons plus en détail plus loin dans ce chapitre.

Ainsi, vous pouvez aussi utiliser la **méthode** appelée `.append()` lorsque vous souhaitez ajouter un seul élément à la fin d'une *liste*.

Voici ce que cela donne dans le cas de l'exemple précédent :

- On crée tout d'abord une liste vide :

```
1 a = []
2
3 a
```

[]

- Puis, on ajoute deux éléments à cette liste créée, l'un après l'autre :

```
1 a.append(15)
2
3 a
```

[15]

```
1 a.append(-5)
2
3 a
```

[15, -5]

Grâce à la méthode `.append()`, on peut donc ajouter des éléments à la fin de la *liste*, comme dans le cas de la *liste* $a = [15, -5]$.

4. Indiçage négatif

4.1. Principes

La *liste* peut également être **indexée** avec des **nombre négatifs** selon le modèle suivant :

Liste:	['girafe', 'tigre', 'singe', 'souris']			
Indice positif:	0	1	2	3
Indice négatif:	-4	-3	-2	-1

Ou encore :

Liste:	['A', 'B', 'C', 'D', 'E', 'F']					
Indice positif:	0	1	2	3	4	5
Indice négatif:	-6	-5	-4	-3	-2	-1

Les **indices négatifs** reviennent à compter à partir de la fin.

A retenir

Un indice strictement négatif dans une liste permet d'accéder à la liste en se référant à la fin de la liste. Par exemple, $S[-5]$ désigne le 5^e élément de S à partir de la fin.

Dans le cas d'une séquence de longueur 8 (c'est juste un exemple), voici une visualisation de la correspondance entre indices négatifs et indices positifs :

	-8	-7	-6	-5	-4	-3	-2	-1	
S									
	0	1	2	3	4	5	6	7	

$S[-5] = S[3]$

4.2. Cas pratiques

Le principal avantage de l'indiçage négatif est que l'on peut accéder au dernier élément d'une *liste* à l'aide de l'**indice** -1 sans pour autant connaître la longueur de cette *liste*. , comme le montre l'exemple suivant :

```
1 animaux = ['girafe', 'tigre', 'singe', 'souris']
1 animaux[-1]
'souris'
```

De même, l'avant-dernier élément a lui l'*indice* -2 , comme le montre l'exemple suivant :

```
1 animaux[-2]
'singe'
```

Pour accéder au premier élément de la liste avec un indice négatif, il faut par contre connaître le bon indice, comme par exemple :

```
1 animaux[-4]
'girafe'
```

Il est donc préférable dans ce cas d'utiliser un indexage positif :

```
1 animaux[0]
'girafe'
```

5. Tranches (ou slides)

5.1. Principes

Un autre avantage des *listes* est la possibilité de sélectionner une partie d'une *liste* en utilisant une approche par **tranche de liste**, aussi appelé **slide**, soit un indexage construit sur le modèle **[m:n+1]** pour récupérer tous les éléments, du *émième* au *énième* (de l'élément **m** inclus à l'élément **n+1** exclu).

On dit alors qu'on récupère une **tranche de la liste**, ou **slide**, comme le montre les nombreux exemples suivants :

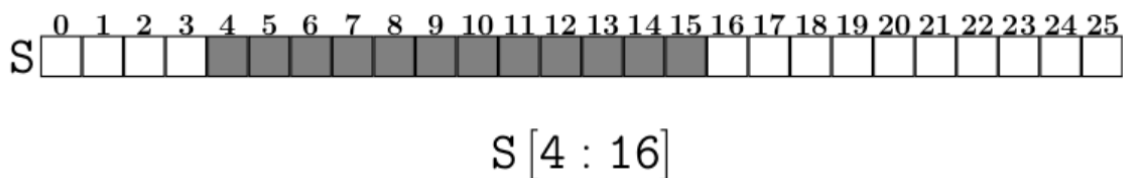
```
1 animaux = ['girafe', 'tigre', 'singe', 'souris']

1 animaux[0:2]
['girafe', 'tigre']

1 animaux[0:3]
['girafe', 'tigre', 'singe']
```

A retenir

Soit **S** une séquence, par exemple une chaîne ou une liste. Une expression de la forme **S[4:16]** est un *slice* dans sa syntaxe de base. Cette syntaxe utilise deux indices, ici les indices 4 (indice de début du slice) et 16 (indice de fin du slice).



5.2. Cas pratiques

Comme nous l'avons compris, la syntaxe des **tranches de liste** ou **slices** permet de *modifier* une *liste* suivant une tranche.

Sachant qu'il existe une multitude de tranchage, il sera plus pratique de résumer dans les tableaux ci-dessous ce que l'on utilise dans 80% de la pratique. On prendra pour exemple le tranchage d'une *liste* de noms nommée **nom** :

	nom = ['eva', 'john', 'jeff', 'yves', 'lina', 'emi', 'oh']						
Indice positif:	0	1	2	3	4	5	6
Indice négatif:	-7	-6	-5	-4	-3	-2	-1

Tableau 1 : Tranchage classique

Lorsqu'aucun indice n'est indiqué à gauche ou à droite du symbole deux-points [**x**:] ou [:**x**], Python prend par défaut tous les éléments depuis le début ou tous les éléments jusqu'à la fin respectivement.

Action	Code	Exécution
Extraction	nom[2:7]	['jeff', 'yves', 'lina', 'emi', 'oh']
Les 4 premiers	nom[:4]	['eva', 'john', 'jeff', 'yves']
Les 4 derniers	nom[-4:]	['yves', 'lina', 'emi', 'oh']
Tous sauf les 4 premiers	nom[4:]	['lina', 'emi', 'oh']
Tous sauf les 4 derniers	nom[:-4]	['eva', 'john', 'jeff']
Tous	nom[:]	['eva', 'john', 'jeff', 'yves', 'lina', 'emi', 'oh']

Tableau 2 : Tranchage par intervalle

On peut aussi préciser le pas en ajoutant un symbole deux-points supplémentaire et en indiquant le pas par un entier [**x**: :] ou [: : **x**].

Action	Code	Exécution
De 3 en 3	nom[::3]	['eva', 'yves', 'oh']
De 3 en 3 à partir de la fin	nom[::-3]	['oh', 'yves', 'eva']
Les indices pairs	nom[::2]	['eva', 'jeff', 'lina', 'oh']
Les indices impairs	nom[1::2]	['john', 'yves', 'emi']
Copie superficielle	nom[:]	['eva', 'john', 'jeff', 'yves', 'lina', 'emi', 'oh']
Copie à l'envers	nom[::-1]	['oh', 'emi', 'lina', 'yves', 'jeff', 'john', 'eva']

Finalement, on se rend compte que l'accès au contenu d'une liste fonctionne sur le modèle **liste[début:fin:pas]**.

6. Fonctions propres aux listes

6.1. Fonction *len()*

L'instruction *len()* vous permet de connaître la longueur d'une *liste*, c'est-à-dire le nombre d'éléments que contient la *liste*.

Voici un exemple d'utilisation :

```
1 animaux = ['girafe', 'tigre', 'singe', 'souris']
2
3 len(animaux)
4
```

6.2. Fonction *list()*

L'instruction *range()* est une fonction spéciale en Python qui génère des nombres entiers compris dans un intervalle. Lorsqu'elle est utilisée en combinaison avec la fonction *list()*, on obtient une liste d'entiers.

Par exemple :

```
1 list(range(10))
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Rappel

La structure syntaxique de la fonction *range()* en Python ne présente aucune difficulté. La fonction accepte **jusqu'à trois paramètres de transfert (start, stop, step)** et vous renvoie une séquence de nombres comme résultat.

```
range(start, stop, step)
```

Il est inutile de spécifier tous les paramètres lors de l'appel de la fonction Python *range*. Seul le **paramètre stop est obligatoire**. Vous utilisez celui-ci pour définir la valeur de fin.

Enfin, prenez garde aux arguments optionnels par défaut (**0 pour début** et **1 pour pas**) :

La commande `list(range(10))` a généré une *liste* contenant tous les nombres entiers de 0 inclus à 10 exclu.

Dans l'exemple ci-dessus, la fonction *range()* a pris un argument, mais elle peut également prendre deux ou trois arguments, voyez plutôt :

```
1 list(range(0,5))
[0, 1, 2, 3, 4]
```

Autres exemples :

```
1 list(range(15,20))
[15, 16, 17, 18, 19]
```



```
1 list(range(0,1000,200))  
[0, 200, 400, 600, 800]  
  
1 list(range(2,-2,-1))  
[2, 1, 0, -1]
```

Cas particulier

Soit le cas particulier :

```
1 list(range(10,0))  
[]
```

Ici la *liste* est vide car Python a pris la valeur du pas par défaut qui est de 1. Ainsi, si on commence à 10 et qu'on avance par pas de 1, on ne pourra jamais atteindre 0. Python génère ainsi une *liste vide*.

Pour éviter ça, il faudrait, par exemple, préciser un pas de *-1* pour obtenir une *liste* d'entiers décroissants :

```
1 list(range(10,0,-1))  
[10, 9, 8, 7, 6, 5, 4, 3, 2, 1]
```

6.3. Fonctions *min()* et *max()*

Les fonctions *min()* et *max()* renvoient respectivement le **minimum** et le **maximum** d'une *liste* passée en argument.

Dans l'exemple suivant, on utilise les fonctions *range()* et *list()* pour générer une liste de nombres entiers, soit :

```
1 liste = list(range(10))  
2  
3 liste  
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]  
  
1 min(liste)  
0  
  
1 max(liste)  
9
```

Même si en théorie ces fonctions peuvent prendre en argument une *liste* de chaînes de caractères (*strings*), on les utilisera la plupart du temps avec des types numériques (*liste* d'entiers et / ou de floats).

Remarque

Il existe également des fonctions *min()*, *max()* pouvant prendre plusieurs arguments **entiers** et / ou **floats**, par exemple :

<pre>1 min(2, 3)</pre>	<pre>1 max(23, 27)</pre>
2	27

Par conséquent, il ne faut **pas mélanger entiers** et *floats* d'une part avec une *liste* d'autre part, car cela renvoie une erreur :

```
1 min(liste, 3, 4)
```

```
-----  
TypeError                                Traceback (most recent call last)  
<ipython-input-63-5e20e04f371d> in <module>  
----> 1 min(liste, 3, 4)  
  
TypeError: '<' not supported between instances of 'int' and 'list'
```

Soit on passe plusieurs *entiers* et / ou *floats* en argument, soit on passe une *liste* unique.

6.4. Fonction *sum()*

La fonction *sum()* renvoie respectivement la **somme** d'une *liste* passée en argument.

Dans l'exemple suivant, on utilise les fonctions *range()* et *list()* pour générer une liste de nombres entiers, soit :

Soit :

```
1 liste = list(range(10))  
2  
3 liste
```

[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

```
1 sum(liste)
```

45

7. Listes de listes

Sachez qu'il est tout à fait possible de construire des **listes de listes**. Cette fonctionnalité peut parfois être très pratique.

Par exemple :

```
1 enclos1 = ['girafe', 4]
2 enclos2 = ['tigre', 2]
3 enclos3 = ['singe', 5]
4 zoo = [enclos1, enclos2, enclos3]
```

```
1 zoo
```

```
[['girafe', 4], ['tigre', 2], ['singe', 5]]
```

Dans cet exemple, chaque *sous-liste* contient une catégorie d'animal et le nombre d'animaux pour chaque catégorie.

Pour accéder à un élément de la *liste*, on utilise l'*indilage* habituel :

```
1 zoo[1]
```

```
['tigre', 2]
```

Pour accéder à un élément de la *sous-liste*, on utilise un **double indilage** :

```
1 zoo[1][0]
```

```
'tigre'
```

```
1 zoo[1][1]
```

```
2
```

Commentaires

On verra un peu plus loin qu'il existe en Python des dictionnaires qui sont également très pratiques pour stocker de l'information structurée.

On verra aussi qu'il existe un module nommé NumPy qui permet de créer des listes ou des tableaux de nombres (vecteurs et matrices) et de les manipuler.

8. Chaînes de caractères et listes

8.1. Principes

Les **chaînes de caractères** peuvent être considérées comme des *listes* (de caractères) un peu particulières.

On prendra par exemple la *chaîne de caractères* **animaux** qui peut être manipulée avec des fonctions et des principes propres aux *listes* :

```
1 animaux = "girafe tigre"
2
3 animaux

'girafe tigre'

1 len(animaux)

12

1 animaux[3]

'a'
```

8.2. Tranches de chaînes de caractères

Nous pouvons donc utiliser certaines propriétés des *listes* comme les **tranches** sur des *chaînes de caractères* :

```
1 animaux = "girafe tigre"

1 animaux[0:4]

'gira'

1 animaux[9:]

'gre'

1 animaux[: -2]

'girafe tig'

1 animaux[1:-2:2]

'iaetg'
```

8.3. Modifier une liste de chaînes de caractères

Règles

A contrario des listes, les chaînes de caractères présentent toutefois une différence notable, ce sont des listes **non modifiables**.

Une fois une chaîne de caractères définie, vous ne pouvez plus modifier un de ses éléments. Le cas échéant, Python renvoie un message d'erreur :

```
1 animaux = "girafe tigre"

1 animaux[4]

'f'

1 animaux[4] = "F"

-----
TypeError                                Traceback (most recent call last)
<ipython-input-76-41b21fc6ce8b> in <module>
----> 1 animaux[4] = "F"

TypeError: 'str' object does not support item assignment
```

Par conséquent, si vous voulez modifier une *chaîne de caractères*, vous devez en **construire une nouvelle**. Pour cela, n'oubliez pas que les opérateurs de *concaténation* (+) et de *duplication* (*) peuvent vous aider. Vous pouvez également générer une *liste*, qui elle est *modifiable*, puis revenir à une *chaîne de caractères*.

9. Méthodes de liste

9.1. Principes

Les *listes* possèdent plusieurs **méthodes** très utiles à leur manipulation, et qu'il est nécessaire de connaître pour utiliser ces structures de données efficacement.

Une méthode est en quelque sorte une fonction qui agit sur la *liste* auquel elle est attachée par un **point** (.).

Par exemple, la méthode `.append()`, qui ajoute un élément à la fin d'une *liste*, comme suit :

```

1 a = [1, 2, 3]
2
3 a.append(5)
4
5 a
[1, 2, 3, 5]
```

9.2. Tableau des principales méthodes

Method	Description	Example
<code>lst.append(x)</code>	Appends element <code>x</code> at the end of the list <code>lst</code>	<pre>>>> l = [] >>> l.append(42) >>> l.append(21) [42, 21]</pre>
<code>lst.clear()</code>	Removes all elements from the list <code>lst</code> —which becomes empty.	<pre>>>> lst = [1, 2, 3, 4, 5] >>> lst.clear() []</pre>
<code>lst.copy()</code>	Returns a copy of the list <code>lst</code> . Copies only the list, not the elements in the list (shallow copy).	<pre>>>> lst = [1, 2, 3] >>> lst.copy() [1, 2, 3]</pre>
<code>lst.count(x)</code>	Counts the number of occurrences of element <code>x</code> in the list <code>lst</code> .	<pre>>>> lst = [1, 2, 42, 2, 1, 42, 42] >>> lst.count(42) 3 >>> lst.count(2) 2</pre>
<code>lst.extend(iter)</code>	Adds all elements of an iterable <code>iter</code> (e.g. another list) to the list <code>lst</code> .	<pre>>>> lst = [1, 2, 3] >>> lst.extend([4, 5, 6]) [1, 2, 3, 4, 5, 6]</pre>
<code>lst.index(x)</code>	Returns the position (index) of the first occurrence of value <code>x</code> in the list <code>lst</code> .	<pre>>>> lst = ["Alice", 42, "Bob", 99] >>> lst.index("Alice") 0 >>> lst.index(99, 1, 3) ValueError: 99 is not in list</pre>

Method	Description	Example
<code>lst.insert(i, x)</code>	Inserts element <code>x</code> at position (index) <code>i</code> in the list <code>lst</code> .	<pre>>>> lst = [1, 2, 3, 4] >>> lst.insert(3, 99) [1, 2, 3, 99, 4]</pre>
<code>lst.pop()</code>	Removes and returns the final element of the list <code>lst</code> .	<pre>>>> lst = [1, 2, 3] >>> lst.pop() 3 >>> lst [1, 2]</pre>
<code>lst.remove(x)</code>	Removes the first occurrence of element <code>x</code> in the list <code>lst</code>	<pre>>>> lst = [1, 2, 99, 4, 99] >>> lst.remove(99) >>> lst [1, 2, 4, 99]</pre>
<code>lst.reverse()</code>	Reverses the order of elements in the list <code>lst</code> .	<pre>>>> lst = [1, 2, 3, 4] >>> lst.reverse() >>> lst [4, 3, 2, 1]</pre>
<code>lst.sort()</code>	Sorts the elements in the list <code>lst</code> in ascending order.	<pre>>>> lst = [88, 12, 42, 11, 2] >>> lst.sort() # [2, 11, 12, 42, 88] >>> lst.sort(key=lambda x: str(x)[0]) # [11, 12, 2, 42, 88]</pre>

9.3. Exemple d'application

À l'origine, on prendra la *liste* :

```
pgm_langs = ['Python', 'Go', 'Rust', 'JavaScript']
```

Cette *liste* sera ensuite modifiée au fur et à mesure de l'application des méthodes qui s'ensuivra.

1) `insert()`

On veut insérer un élément à un *indice* (*i*) particulier. Pour ce faire, on va utiliser la méthode `insert()`, laquelle prend en compte :

- l'*indice* (*i*) auquel l'*élément* doit être inséré, et
- l'*élément* à insérer.

2) `append()`

On veut ajouter un *élément* à la fin de la *liste*. Pour ce faire, on va utiliser la méthode `append()`.

3) `extend()`

On peut utiliser la méthode `append()` pour ajouter un seul élément à une *liste*. Mais que se passe-t-il si l'on souhaite ajouter plusieurs éléments à une *liste* existante ? La méthode `extend()` fournit une syntaxe concise pour le faire.

4) `reverse()`

On veut renverser l'ordre des éléments d'une liste. Pour ce faire, on va utiliser la méthode `reverse()`.

5) `sort()`

On veut trier les éléments d'une *liste*. Pour ce faire, on va utiliser la méthode `sort()` (ordre alphabétique par défaut).

6) `count()`

On veut connaître le nombre de fois qu'un élément particulier apparaît dans une *liste*. Pour ce faire, on va utiliser la méthode `count()`, laquelle renvoie le nombre de fois qu'un élément particulier apparaît dans une *liste*.

Réponse

1) `insert(1, 'Scala')`

Insérer 'Scala' à l'indice `i = 1` de la liste `pgm_langs` en utilisant la méthode `insert()`.

```
pgm_langs = ['Python', 'Go', 'Rust', 'JavaScript']
pgm_langs.insert(1, 'Scala')
print(pgm_langs)
# Output: ['Python', 'Scala', 'Go', 'Rust', 'JavaScript']
```

2) `append('Java')`

Ajouter 'Java' à la fin de la liste `pgm_langs` en utilisant la méthode `append()`.

```
pgm_langs.append('Java')
print(pgm_langs)
# Output: ['Python', 'Scala', 'Go', 'Rust', 'JavaScript', 'Java']
```

3) `extend(more_langs)`

Ajouter les éléments de la liste `more_langs` à la `pgm_langs` liste à l'aide de la méthode `extend()`.

```
more_langs = ['C++', 'C#', 'C']
pgm_langs.extend(more_langs)
print(pgm_langs)
# Output: ['Python', 'Scala', 'Go', 'Rust', 'JavaScript', 'Java', 'C++', 'C#', 'C']
```

4) `reverse()`

Renverser les éléments de la liste `pgm_langs` à l'aide de la méthode `reverse()`

```
pgm_langs.reverse()
print(pgm_langs)
# Output: ['C', 'C#', 'C++', 'Java', 'JavaScript', 'Rust', 'Go', 'Scala', 'Python']
```


5) `sort()`

Trier par ordre alphabétique les éléments de la liste `pgm_langs` à l'aide de la méthode `sort()`.

```
pgm_langs.sort()  
print(pgm_langs)  
# Output: ['C', 'C#', 'C++', 'Go', 'Java', 'JavaScript', 'Python', 'Rust', 'Scala']
```

6) `count('Go')`

Donner le nombre de fois que l'élément `'Go'` apparaît dans la liste `pgm_langs` à l'aide de la méthode `count()`.

```
print(pgm_langs.count('Go'))  
# Output: 1
```

10. Exercices

1. Accéder aux éléments d'une liste

On donne la liste suivante :

```
L = [10, 20, 30, 40, 50]
```

Afficher :

- le premier élément de la liste ;
- le dernier élément ;
- les trois premiers éléments.

2. Modifier une liste

Avec la liste :

```
L = [1, 2, 3, 4]
```

Effectuer les opérations suivantes:

- remplacer le 2 par 50 ;
- ajouter 5 à la fin de la liste ;
- insérer 0 au début de la liste.

3. Calculs sur une liste

Avec la liste :

```
L = [4, 7, 1, 3, 9]
```

Calculer sur la liste :

- la somme des éléments ;
- la moyenne des éléments ;
- le maximum des éléments.

4. Parcours d'une liste

Afficher chaque élément de la liste suivante sur une nouvelle ligne :

```
L = ["pomme", "banane", "orange"]
```

Tel que :

```
pomme  
banane  
orange
```

5. Compréhension de liste

A partir de la liste suivante :

```
L = [1, 2, 3, 4, 5]
```

Construire une nouvelle liste contenant le carré de chaque élément de la liste.

6. Filtrer une liste

A partir de la liste suivante :

```
L = [12, 5, 18, 7, 3, 20]
```

Créer une nouvelle liste contenant uniquement les nombres pairs.

7. Trouver un élément dans une liste

A partir de la liste suivante :

```
L = ["Python", "Java", "C++", "Rust"]
```

Vérifier si "Java" est présent, et afficher un message adapté.

8. Fusion de deux listes

Fusionner les listes suivantes en une seule liste :

```
A = [1, 2, 3]
```

```
B = [4, 5, 6]
```

9. Trier une liste

Trier la liste suivante en ordre croissant puis décroissant :

```
L = [9, 1, 8, 2, 7, 3]
```

10. Supprimer des éléments d'une liste

A partir de la liste suivante :

```
L = [3, 6, 3, 9, 3, 12]
```

Supprimer toutes les occurrences de 3.

Correction

1. Accéder aux éléments d'une liste

```
L = [10, 20, 30, 40, 50]

print(L[0])      # Premier
print(L[-1])     # Dernier
print(L[:3])     # Trois premiers
```

2. Modifier une liste

```
L = [1, 2, 3, 4]

L[1] = 20
L.append(5)
L.insert(0, 0)

print(L) # [0, 20, 3, 4, 5]
```

3. Calculs sur une liste

```
L = [4, 7, 1, 3, 9]

somme = sum(L)
moyenne = sum(L) / len(L)
maximum = max(L)

print(somme, moyenne, maximum)
```

4. Parcours d'une liste

```
L = ["pomme", "banane", "orange"]

for fruit in L:
    print(fruit)
```

5. Compréhension de liste

```
L = [1, 2, 3, 4, 5]
carres = [x*x for x in L]
print(carres)
```

6. Filtrer une liste

```
L = [12, 5, 18, 7, 3, 20]
pairs = [x for x in L if x % 2 == 0]
print(pairs) # [12, 18, 20]
```

7. Trouver un élément dans une liste

```
L = ["Python", "Java", "C++", "Rust"]

if "Java" in L:
    print("Java est présent.")
else:
    print("Java n'est pas présent.")
```

8. Fusion de deux listes

```
A = [1, 2, 3]
B = [4, 5, 6]

C = A + B
print(C)
```

9. Trier une liste

```
L = [9, 1, 8, 2, 7, 3]

L_asc = sorted(L)
L_desc = sorted(L, reverse=True)

print(L_asc)
print(L_desc)
```

10. Supprimer des éléments d'une liste

```
L = [3, 6, 3, 9, 3, 12]

L = [x for x in L if x != 3]
print(L) # [6, 9, 12]
```